

PROVISIONAL PATENT APPLICATION

METHOD AND SYSTEM FOR VERIFIABLE TRUST COORDINATION AMONG AUTONOMOUS COMPUTATIONAL AGENTS

自主計算代理間之可驗證信任協調方法與系統

Inventor: [Inventor Name]

Date of Filing: [Filing Date]

Prepared for: Saint Island International Patent & Law Offices

聖島國際專利商標聯合事務所

ATTORNEY-CLIENT PRIVILEGED & CONFIDENTIAL

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is related to U.S. Patent Application Serial No. [Patent #1] ("Method and System for Gaming Random Number Generation" — Trust Source), U.S. Patent Application Serial No. [Patent #2] ("VDF-Protected Batch Random Number Generation with Anti-Preview Guarantees" — Trust Integrity), and U.S. Patent Application Serial No. [Patent #3] ("Generating, Scoring, Signing, Anchoring, and Verifying Verifiable Trust Artifacts" — Trust Evidence).

[0002] The present invention provides the Trust Coordination layer (Layer 4) of the Computational Trust Infrastructure. Where Patents #1–#3 address the generation, integrity, and packaging of trust for individual computations, the present invention addresses a distinct problem: **how trust propagates between multiple autonomous agents and across system boundaries**.

[0003] The present invention is a protocol for trust transmission — analogous to TCP for data transmission. It does not control what agents do; it ensures that trust propagates correctly between them.

FIELD OF THE INVENTION

[0004] The present invention relates to systems and methods for coordinating, propagating, and verifying trust among autonomous computational agents operating in multi-agent systems. More specifically, the invention relates to: trust chain construction through directed acyclic graph (DAG) lineage linking; multi-agent trust consensus protocols; cross-organizational trust handoff mechanisms; and trust propagation invariant enforcement.

BACKGROUND OF THE INVENTION

The Multi-Agent Trust Problem

[0005] Modern AI systems are increasingly multi-agent: swarms of autonomous agents collaborate, delegate, and hand off tasks. When Agent A assigns a task to Agent B based on a non-deterministic process (e.g., using a Verifiable Entropy Object as described in Patent #3), Agent B needs to trust that the assignment was fair. Agent C (an auditor or supervisor) needs to verify the entire chain of decisions — from the root allocation to each sub-task assignment.

[0006] This problem does not exist in single-agent systems, where trust terminates at one entity. But in multi-agent systems — AI agent swarms, robotic fleets, distributed enterprise workflows, cross-organizational AI pipelines — trust must *flow* between agents, and each link in the flow must be independently verifiable.

[0007] No existing system provides a protocol for verifiable trust propagation between autonomous agents. Specifically:

[0008] **Multi-agent frameworks (LangChain, CrewAI, AutoGPT):** These frameworks orchestrate agent collaboration but do not provide verifiable trust chains. Task assignments are logged but not cryptographically linked. An auditor cannot independently verify that a specific agent received a specific task based on specific, quality-scored entropy.

[0009] **Blockchain consensus:** Blockchain achieves consensus on shared state but does not address trust in off-chain agent decisions. A blockchain can record that a decision was made; it cannot verify that the decision process — including its non-deterministic inputs — was trustworthy.

[0010] PKI and certificate chains: Public Key Infrastructure provides identity chains (certificate authority → intermediate → end entity). However, PKI chains verify identity, not computational trust. A valid certificate does not prove that the certificate holder's computation was fair or that its non-deterministic inputs were quality-scored.

[0011] There exists a need for a protocol that enables trust to be constructed, propagated, and verified across autonomous agents — ensuring that trust chains are independently auditable, trust does not amplify through propagation, and trust degradation is transparent.

SUMMARY OF THE INVENTION

[0012] The present invention provides:

1. **Trust Chain Construction:** A method for linking child trust artifacts to parent trust artifacts via cryptographic lineage references, creating directed acyclic graphs (DAGs) of trust that enable complete decision-tree reconstruction and independent verification.
2. **Multi-Agent Trust Consensus:** A protocol for combining trust artifacts from multiple autonomous agents into a jointly verifiable consensus trust artifact, where each participant's contribution is recorded and independently verifiable.
3. **Cross-Boundary Trust Handoff:** A mechanism for packaging, transmitting, and verifying trust assertions across organizational trust boundaries without requiring mutual trust between organizations.
4. **Trust Propagation Invariants:** Enforcement mechanisms guaranteeing that trust does not amplify (child trust $\leq$ parent trust), degradation is transparent, and every link is independently verifiable.

5. **Trust Chain Verification:** A method for traversing and independently verifying an entire trust chain from any leaf to the root, without trusting any intermediate node.
-

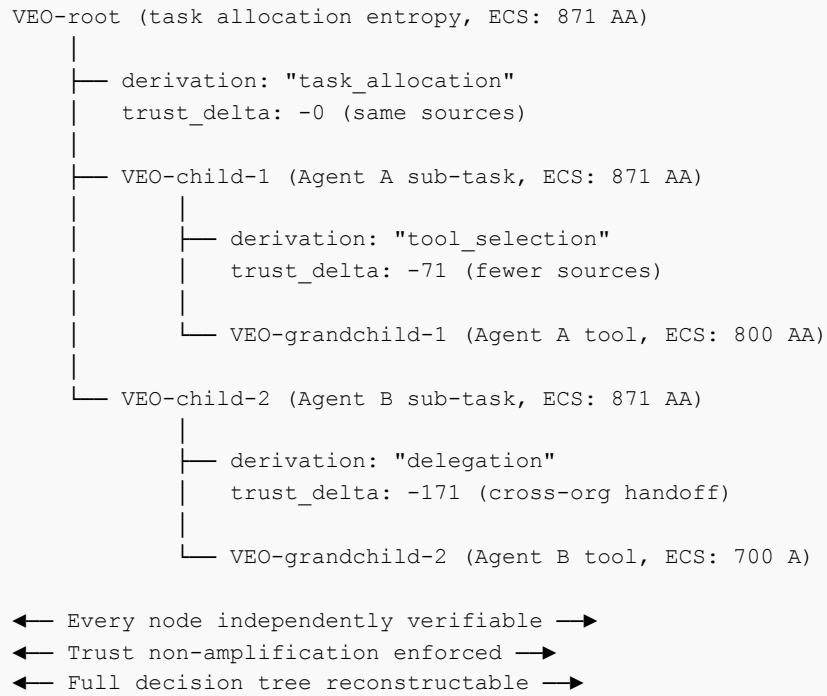
DETAILED DESCRIPTION OF PREFERRED EMBODIMENTS

1. Trust Chain Construction

[0013] When a trust artifact (e.g., a VEO as described in Patent #3) is used to make a decision that spawns child computations, the system constructs a trust chain by linking child trust artifacts to their parent via a lineage record. The lineage record contains:

- a. **Parent object identifier(s):** One or more identifiers referencing the trust artifacts that contributed to the current computation.
- b. **Lineage hash:** A cryptographic hash computed over the parent identifiers and the current artifact's entropy, enabling tamper detection in the lineage chain.
- c. **Derivation type:** The relationship between parent and child (e.g., "task_allocation", "sub_task", "delegation", "consensus_input").
- d. **Trust delta:** A record of how the trust level changed from parent to child (e.g., ECS score reduction due to fewer sources, additional processing, or time decay).

FIG. 1 — Trust Chain (DAG) Construction



[0014] The trust chain forms a directed acyclic graph (DAG) — not a linear chain — because a single parent VEO may have multiple children (e.g., a task allocation VEO spawns multiple sub-task VEOs) and a child VEO may have multiple parents (e.g., a consensus VEO combines entropy from multiple participants).

2. Multi-Agent Trust Consensus

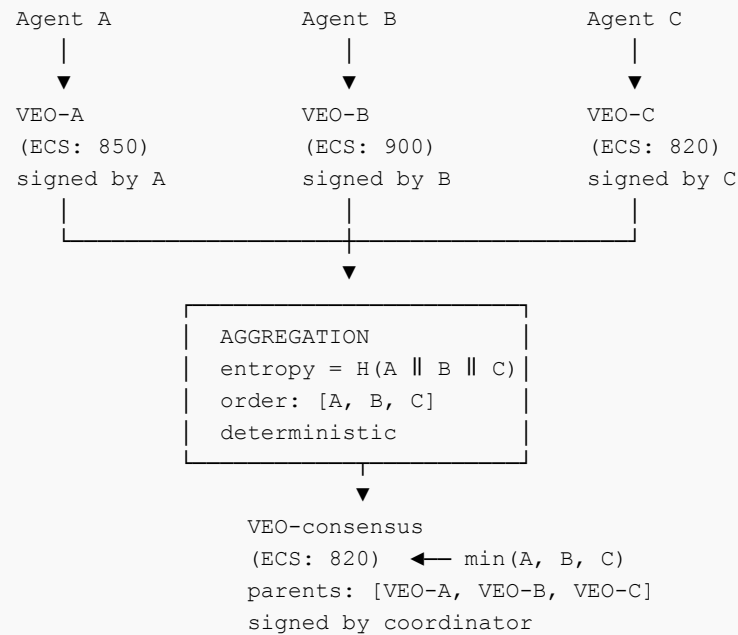
[0015] When multiple autonomous agents must agree on a shared non-deterministic value — for example, jointly selecting a leader, allocating a contested resource, or establishing task order — the system provides a multi-agent trust consensus protocol:

- a. **Contribution phase:** Each participating agent generates or provides a trust artifact (VEO) containing independently sourced entropy. Each contribution is signed by its

respective agent.

- b. **Aggregation phase:** The contributions are collected and combined using a deterministic aggregation function (e.g., concatenation + cryptographic hash of all participant entropies in a defined order). The aggregation is deterministic: given the same inputs in the same order, any party produces the same output.
- c. **Consensus artifact construction:** A new consensus trust artifact is constructed. This artifact references all contributing artifacts as parents (via the lineage mechanism), carries a composite ECS score derived from the contributors' scores, and is signed by the coordinating entity (or jointly signed by participants in a threshold-signature variant).
- d. **Independent verification:** Any party can verify the consensus artifact by: (i) verifying each contributing artifact independently, (ii) reproducing the aggregation from the contributing entropies, (iii) confirming the consensus entropy matches the reproduced aggregation, and (iv) verifying the consensus artifact's signature.

FIG. 2 — Multi-Agent Trust Consensus



- Any party can:
1. Verify VEO-A, VEO-B, VEO-C independently
 2. Reproduce $H(A \parallel B \parallel C)$
 3. Confirm VEO-consensus entropy matches
 4. Verify coordinator signature

3. Cross-Boundary Trust Handoff

[0016] When trust must cross organizational boundaries — for example, an enterprise AI system delegating to a third-party agent service, or a consortium of organizations sharing trusted computation results — the system provides a cross-boundary trust handoff protocol:

- a. Trust package construction:** The sending organization constructs a trust handoff package containing: the trust artifact (VEO), the sender's signature, the sender's public key or certificate, the handoff policy (specifying what the receiver may and may not do with the trust artifact), and a handoff identifier for tracking.

- b. **Transmission:** The trust package is transmitted through any communication channel (API, message queue, direct connection). The package is self-verifying — no trust in the transmission channel is required.
- c. **Receiving verification:** The receiving organization verifies the trust package by: (i) verifying the trust artifact's provider signature (confirming the VEO was issued by a recognized trust source), (ii) verifying the sender's handoff signature (confirming the VEO was transmitted by a specific party), (iii) evaluating the handoff policy, and (iv) optionally re-scoring the ECS based on the receiver's own policy requirements.
- d. **Handoff attestation:** The handoff event is recorded by both parties as a new lineage entry, creating an auditable record that trust crossed organizational boundaries at a specific time, with specific policy constraints.

[0017] The handoff protocol ensures that trust does not require mutual organizational trust. Organization A and Organization B need not trust each other. They need only trust the cryptographic proofs in the trust artifacts and the mathematical properties of the propagation protocol.

4. Trust Propagation Invariants

[0018] The system enforces three invariants that govern all trust propagation:

[0019] **Invariant 1: Non-amplification.** A child trust artifact's quality score (ECS) cannot exceed the minimum quality score of its parent artifacts. Trust can only degrade or remain constant through propagation — never increase. This is enforced computationally: the system caps the child's ECS at $\min(\text{parent ECS values})$. If a child computation uses fewer entropy sources or introduces additional processing, the ECS decreases accordingly.

[0020] Invariant 2: Transparent degradation. When trust degrades through propagation (due to fewer sources, cross-boundary handoff, time decay, or processing overhead), the degradation is explicitly recorded in the trust delta field of the lineage record. There are no hidden trust losses. An auditor examining any node in the trust chain can see exactly why its trust level differs from its parent's.

[0021] Invariant 3: Independent verifiability. Every node in a trust chain is independently verifiable without trusting any other node. A verifier can check any trust artifact's integrity, signature, ECS, and lineage references without cooperating with the artifact's generator, the parent artifact's generator, or any intermediate node in the chain.

5. Trust Chain Verification (Traversal)

[0022] The system provides a verification algorithm for traversing and verifying an entire trust chain:

- a. **Leaf verification:** Verify the target trust artifact (integrity, signature, ECS, policy compliance).
- b. **Lineage traversal:** Follow the lineage references to all parent artifacts.
- c. **Parent verification:** Verify each parent artifact independently.
- d. **Invariant check:** Confirm that the child's $ECS \leq \min(\text{parent ECS values})$.
- e. **Trust delta check:** Confirm that the recorded trust delta correctly explains the score difference between parent and child.
- f. **Recursive traversal:** Repeat steps 2–5 for each parent until root artifacts (those with no parents) are reached.
- g. **Root verification:** Verify root artifacts against their entropy sources (drand beacon signatures, blockchain anchors, etc.).

[0023] The result is a complete, independently verified trust chain from leaf to root. If any verification check fails at any node, the failure is reported with the specific node and check that failed, enabling precise diagnosis.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 — Trust chain (DAG) construction showing parent-child lineage relationships with trust deltas.

FIG. 2 — Multi-agent trust consensus protocol showing contribution, aggregation, and consensus artifact construction.

FIG. 3 — Cross-boundary trust handoff showing package construction, transmission, and receiving verification.

CLAIMS

What is claimed is:

1. A computer-implemented method for coordinating trust among a plurality of autonomous computational agents, comprising:
 - (a) generating, by a first agent, a first trust artifact comprising an entropy payload, source provenance, a quality score, and a cryptographic signature;
 - (b) using the first trust artifact to make a computational decision that spawns one or more child computations;

(c) generating a second trust artifact for a child computation, wherein the second trust artifact includes a lineage record referencing the first trust artifact by identifier;

(d) enforcing a non-amplification invariant wherein the second trust artifact's quality score does not exceed the first trust artifact's quality score; and

(e) enabling independent verification of the trust chain from the second trust artifact to the first trust artifact without requiring cooperation from any agent.

2. The method of claim 1, wherein the lineage record further comprises a lineage hash computed over the parent artifact identifier and the child artifact's entropy, a derivation type indicating the relationship between parent and child, and a trust delta recording the quality score change.

3. The method of claim 1, wherein the trust chain forms a directed acyclic graph enabling a single parent trust artifact to have multiple children and a single child trust artifact to have multiple parents.

4. A computer-implemented method for achieving trust consensus among a plurality of autonomous agents, comprising:

(a) receiving, from each of a plurality of participating agents, a respective trust artifact comprising independently sourced entropy and a cryptographic signature;

(b) combining the entropies from the plurality of trust artifacts using a deterministic aggregation function to produce a consensus entropy;

(c) constructing a consensus trust artifact that references all contributing trust artifacts as parents, carries the consensus entropy, and has a quality score not exceeding the minimum quality score among the contributing artifacts; and

(d) enabling any party to independently verify the consensus by verifying each contributing artifact, reproducing the aggregation, and confirming the consensus entropy matches.

5. The method of claim 4, wherein the deterministic aggregation function comprises concatenating the entropies from the contributing artifacts in a defined order and computing a cryptographic hash of the concatenation.

6. A computer-implemented method for trust handoff across organizational boundaries, comprising:

(a) constructing, by a sending organization, a trust handoff package comprising a trust artifact, a sender signature, a handoff policy, and a handoff identifier;

(b) transmitting the trust handoff package to a receiving organization;

(c) verifying, by the receiving organization, the trust artifact's provider signature and the sender's handoff signature without requiring mutual trust between the organizations; and

(d) recording the handoff event as a lineage entry in both the sending and receiving organizations' trust records.

7. The method of claim 6, wherein the handoff policy specifies constraints on the receiving organization's use of the trust artifact, including permitted derivation types, maximum trust chain depth extension, and re-scoring requirements.

8. A system for verifiable trust coordination among autonomous computational agents, comprising:

one or more processors; and

memory storing instructions that, when executed, cause the system to:

- (a) construct trust chains by linking child trust artifacts to parent trust artifacts via cryptographic lineage references, forming directed acyclic graphs;
- (b) enforce trust propagation invariants including non-amplification, transparent degradation, and independent verifiability at each node;
- (c) combine trust artifacts from multiple agents into consensus trust artifacts using deterministic aggregation; and
- (d) enable trust handoff across organizational boundaries through self-verifying trust packages.

9. The system of claim 8, further comprising a trust chain verification module that traverses a trust chain from any leaf artifact to the root, independently verifying each node's integrity, signature, quality score, and lineage references.

10. A computer-implemented method for verifying a trust chain, comprising:

- (a) receiving a target trust artifact at a leaf position in a trust chain;
- (b) verifying the target artifact's integrity, cryptographic signature, and quality score;
- (c) traversing the target artifact's lineage references to identify parent artifacts;
- (d) for each parent artifact, independently verifying integrity, signature, and quality score;
- (e) confirming that the target artifact's quality score does not exceed the minimum parent quality score;
- (f) recursively repeating steps (c) through (e) for each parent until root artifacts are reached; and
- (g) verifying root artifacts against their original entropy sources,

wherein the entire verification is performed without cooperation from any artifact generator or intermediate node.

ABSTRACT

A method and system for coordinating, propagating, and verifying trust among autonomous computational agents in multi-agent systems. The system constructs trust chains by linking child trust artifacts to parent trust artifacts via cryptographic lineage references, forming directed acyclic graphs (DAGs) enabling complete decision-tree reconstruction. A multi-agent trust consensus protocol combines trust artifacts from multiple agents into jointly verifiable consensus artifacts using deterministic aggregation. A cross-boundary trust handoff mechanism enables trust to propagate across organizational boundaries through self-verifying trust packages without requiring mutual organizational trust. Three propagation invariants are enforced: non-amplification (child trust $\leq$ parent trust), transparent degradation (all trust changes are recorded), and independent verifiability (every node is verifiable without trusting any other node). A trust chain verification algorithm enables traversal and independent verification of an entire chain from any leaf to the root.